CENTRUM PROJEKTOWE

Dokumentacja oficjalna

Stan projektu, wnioski z architektury Po Kapiemu, kierunek Raspberry Pi Hub oraz etapowy plan prac 7–15.

local-first | prywatna pamięć projektowa | Raspberry Pi + NVMe | integracje kontrolowane

Wersja: 2026-10-08
Status: po Etapie 6
Repozytorium: kapi1944/Centrum_Projektowe

Centrum Projektowe ��� dokumentacja oficjalna

Centrum Projektowe — dokumentacja oficjalna

Spis treści

1. Cel i zakres Centrum Projektowego
2. Stan bieżący — co już działa
   - 2.1. Stan potwierdzony w repozytorium
   - 2.2. Funkcjonalny rdzeń dostępny po Etapie 6
   - 2.3. Najważniejsza cecha obecnej architektury
3. Czego nauczyło nas Po Kapiemu
4. Co warto zaczerpnąć z Po Kapiemu
   - 4.1. Macierz decyzji
   - 4.2. Wzorzec 1 — shell aplikacji i nawigacja
   - 4.3. Wzorzec 2 — szybkie wyszukiwanie
   - 4.4. Wzorzec 3 — rejestr sekcji projektu
   - 4.5. Wzorzec 4 — status, postęp i dojrzałość
   - 4.6. Wzorzec 5 — backend auth i bezpieczeństwo
   - 4.7. Wzorzec 6 — Docker, healthcheck i CI
   - 4.8. Wzorzec 7 — Ogarniacz: outbox, retry i idempotencja
5. Co należy odrzucić albo tylko potraktować jako inspirację
   - 5.1. Funkcje społecznościowe
   - 5.2. Recenzje, testy i porównania
   - 5.3. Statyczne `siteData` i `projectDetails`
   - 5.4. localStorage jako źródło prawdy
   - 5.5. Publiczny Caddy + domena jako wymóg
6. Nowa architektura docelowa Centrum Projektowego
   - 6.1. Zasada nadrzędna
   - 6.2. Warstwy
7. Model plików i storage — odejście od Google Drive jako założenia
   - 7.1. Google Drive nie może być typem domenowym pierwszej klasy
   - 7.2. Proponowany model `StorageProvider`
   - 7.3. Metadane pliku
   - 7.4. Plik ≠ dokumentacja

8. Raspberry Pi jako przyszły Centrum Hub
8.1. Rola
8.2. Co warto zaczerpnąć z backendu Po Kapiemu
8.3. Czego nie kopiować automatycznie
9. Zasady integracji z Ogarniaczem i Po Kapiemu
9.1. Ogarniacz
9.2. Po Kapiemu
10. Zaktualizowany etapowy plan prac 7–15
Etap 7 — Pliki, dokumenty i abstrakcja StorageProvider
Etap 8 — Modułowa karta projektu, zasoby, kompetencje i globalne wyszukiwanie
Etap 9 — Stan wejściowy projektu / Inkubator
Etap 10 — Karta zdrowia + Centrum uwagi
Etap 11 — Raspberry Pi Hub — fundament backendu
Etap 12 — Synchronizacja local-first ↔ Hub
Etap 13 — GitHub i providerzy zewnętrzni
Etap 14 — AI + Ogarniacz + Po Kapiemu
Etap 15 — Hardening i wydanie
11. Zasady techniczne i antycele
11.1. Zasady
11.2. Antycele
12. Kryteria gotowości
Po Etapie 8
Po Etapie 10
Po Etapie 12
Po Etapie 15
13. Pytania decyzyjne — architektura danych i storage
P1. Jaką docelową rolę ma pełnić Raspberry Pi?
P2. Co ma być źródłem prawdy dla danych domenowych podczas najbliższych etapów?
P3. Jak fizycznie przechowywać pliki na Raspberry Pi?
P4. Jaką bazę metadanych docelowo uruchomić na Raspberry Pi Hub?
P5. Jak wdrażać usługi na Raspberry Pi?
P6. Jak uzyskiwać dostęp do Hubu spoza domu?
P7. Czy pliki z SSD mają być dostępne również poza interfejsem Centrum?
14. Pytania decyzyjne — UX, bezpieczeństwo i integracje
P8. Jak daleko kopiować interfejsowe wzorce z Po Kapiemu?
P9. Jaki model dostępu zastosować, gdy powstanie Hub?
P10. Jak traktować układ sekcji projektu?
P11. Jak prowadzić integrację z Ogarniaczem?
P12. Jak realizować publikację do Po Kapiemu?

P13. Jak wykonywać kopię SSD z Raspberry Pi?
P14. Jak podchodzić do AI na Raspberry Pi?
Źródła i materiały porównawcze

Centrum Projektowe — dokumentacja oficjalna

Stan projektu, architektura docelowa, wnioski z Po Kapiemu i plan etapowy
Wersja dokumentu: 2026-10-08
Status: dokument kierunkowy po Etapie 6
Repozytorium: `kapi1944/Centrum_Projektowe`

> Ten dokument zastępuje wcześniejsze rozproszone notatki kierunkowe dotyczące etapów 7–15. Uwzględnia rzeczywisty stan Centrum Projektowego, mapę architektury Po Kapiemu, analizę repozytorium `kapi1944/Po_Kapiemu` oraz nowy kierunek: prywatny hub na Raspberry Pi z zewnętrznym SSD NVMe jako przyszła warstwa przechowywania plików i synchronizacji.

Spis treści

1. Cel i zakres Centrum Projektowego
2. Stan bieżący — co już działa
3. Czego nauczyło nas Po Kapiemu
4. Co warto zaczerpnąć z Po Kapiemu
5. Co należy odrzucić albo tylko potraktować jako inspirację
6. Nowa architektura docelowa Centrum Projektowego
7. Model plików i storage — odejście od Google Drive jako założenia
8. Raspberry Pi jako przyszły Centrum Hub
9. Zasady integracji z Ogarniaczem i Po Kapiemu
10. Zaktualizowany etapowy plan prac 7–15
11. Zasady techniczne i antycele
12. Kryteria gotowości
13. Pytania decyzyjne — architektura danych i storage
14. Pytania decyzyjne — UX, bezpieczeństwo i integracje

---

1. Cel i zakres Centrum Projektowego

Centrum Projektowe nie jest kolejnym klasycznym menedżerem zadań. Jego podstawową rolą jest bycie **prywatną pamięcią projektową i warstwą operacyjno-analityczną nad wieloma równoległymi przedsięwzięciami**.

Najważniejszy przepływ pozostaje niezmienny:

```
surowy wpis / pomysł
        ↓
zachowanie oryginału
        ↓
analiza i klasyfikacja
        ↓
weryfikacja przez użytkownika
        ↓
decyzje / pytania / działania / wpływ
        ↓
aktualny model projektu
        ↓
historia zmian i możliwość odpowiedzi „dlaczego?”
```

Centrum powinno odpowiadać na pięć praktycznych pytań:

1. **Gdzie skończyłem?**
2. **Co zostało ustalone i co nadal obowiązuje?**
3. **Co nie zostało wdrożone?**
4. **Co blokuje dalszy postęp?**
5. **Co jest teraz najrozsądniejszym następnym krokiem?**

Publiczne prezentowanie projektów pozostaje domeną **Po Kapiemu**. Ogarniacz pozostaje warstwą przypomnień i codziennego wykonywania zadań. GitHub pozostaje źródłem prawdy dla kodu. Centrum spina te światy, ale nie powinno bez potrzeby przejmować ich ról.

---

2. Stan bieżący — co już działa

2.1. Stan potwierdzony w repozytorium

Publiczny `main` repozytorium Centrum Projektowego zawiera wykonane etapy do modelu realizacji projektu, z ostatnim wypchniętym commitem:

`53d6c28 — feat: add project execution model`

Dodatkowo lokalnie wykonano Etap 6:

`5c6acd8 — feat: add backup and restore`

Etap 6 został zakończony bez push. Raport wykonawczy potwierdza:

- 97/97 testów PASS,
- typecheck PASS,
- lint PASS,
- build PASS,
- `git diff --check` PASS,
- atomowy import i rollback,
- eksport wszystkich 11 istniejących magazynów danych,
- walidację rekordów, kluczy i relacji,
- dwa tryby importu: połączenie i zastąpienie,
- wykrywanie konfliktów,
- brak migracji IndexedDB, ponieważ backup nie wymagał zmiany schematu.

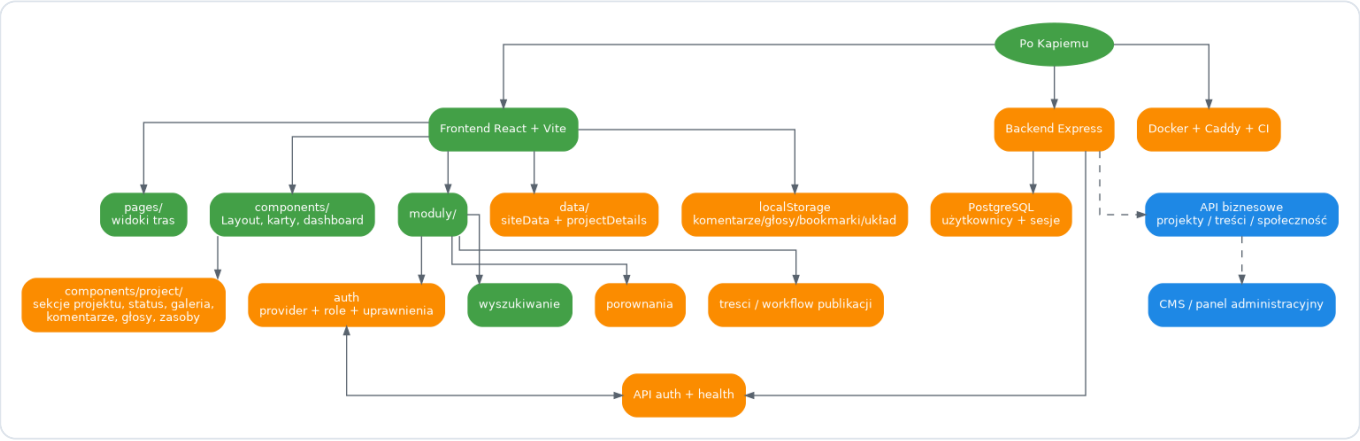
2.2. Funkcjonalny rdzeń dostępny po Etapie 6

Obszar	Stan	Znaczenie
Projekty	✅	tworzenie, edycja, statusy, archiwizacja
Skrzynka / szybki wpis	✅	zachowanie surowej treści bez nadpisywania
„Gdzie skończyłem?”	✅	aktualny stan, ostatnia praca, następny krok
Historia	✅	`ActivityEvent`, zapis ważnych zmian
Analiza wpisu	✅	osobna `CaptureAnalysis`, review przed apply
Decyzje	✅	`DEC-XXXX`, statusy, wiele projektów
Zastępowanie decyzji	✅	historia pozostaje, stara decyzja nie znika
Analiza wpływu	✅	propozycje zmian wymagają zatwierdzenia
Obszary i etapy	✅	struktura wykonawcza projektu

Obszar	Stan	Znaczenie
Elementy pracy	✅	zadanie, research, eksperyment, kontakt, zakup, follow-up, oczekiwanie
Otwarte pytania	✅	osobna encja, nie zwykłe TODO
Blokady	✅	osobna encja z ważnością
Backup / restore	✅ lokalnie	eksport/import pełnej bazy, konflikt + rollback
Repozytoria / dokumenty / zasoby	⏳	następny większy obszar
Raspberry Pi Hub	💡	kierunek docelowy po ustaleniu architektury
Synchronizacja urządzeń	💡	późniejszy etap, nie wdrażać pochopnie

2.3. Najważniejsza cecha obecnej architektury

Centrum jest nadal **local-first**. Dane domenowe są przechowywane w IndexedDB, a logika domenowa jest oddzielona od warstwy persistence. To jest bardzo dobry punkt wyjścia pod późniejszą synchronizację — pod warunkiem, że nie zaczniemy teraz traktować jednego zewnętrznego serwera jako obowiązkowego źródła prawdy.



Po Kapiemu — drzewo struktury technicznej. Najcenniejsze wzorce dla Centrum: oddzielenie frontendu, auth, backendu, bazy i warstwy wdrożeniowej.

3. Czego nauczyło nas Po Kapiemu

Po Kapiemu jest wartościowym „dawcą wzorców”, ale **nie powinno być kopiowane jako całość**.

Mapa architektury Po Kapiemu wskazuje cztery ważne warstwy:

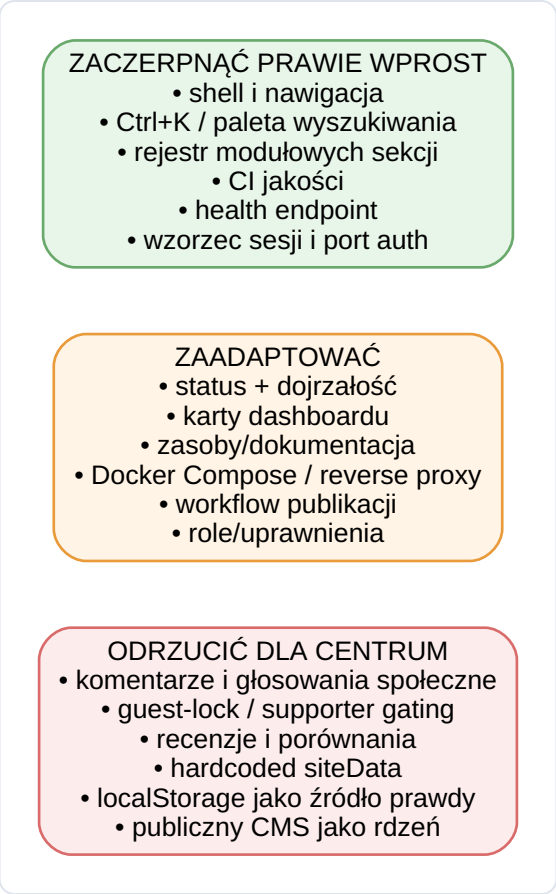
- rozbudowaną nawigację i dashboard,
- frontend React + Vite,
- backend Express + PostgreSQL,
- warstwę wdrożeniową Docker + Caddy + CI.

Repozytorium potwierdza, że te elementy nie są jedynie planem. Istnieją rzeczywiste komponenty i moduły dla:

- layoutu i responsywnej nawigacji,
- szybkiego wyszukiwania z `Ctrl+K`,
- autoryzacji i uprawnień,
- statusu i dojrzałości projektu,
- modularnego rejestru sekcji projektu,
- aktualizacji projektu i zasobów,
- planowania treści,
- Express API, sesji i healthchecku,
- PostgreSQL,
- Docker Compose,
- Caddy,
- GitHub Actions.

Jednocześnie mapa architektury sama wskazuje główny problem Po Kapiemu: część funkcji działa nadal lokalnie w przeglądarce, podczas gdy backend przechowuje głównie użytkowników i sesje. Dokumentacja rekomenduje najpierw zbudowanie jednej warstwy danych biznesowych, a dopiero potem dalsze integracje i administrację.

To doświadczenie jest szczególnie ważne dla Centrum: **nie wolno powtórzyć rozjazdu między lokalnymi danymi interfejsu a serwerem, który zna jedynie część domeny.**



Macierz zapożyczeń: co przenosimy jako wzorzec, co adaptujemy i co świadomie odrzucamy.

4. Co warto zaczerpnąć z Po Kapiemu

4.1. Macierz decyzji

Element Po Kapiemu	Ocena dla Centrum	Decyzja
Shell aplikacji: sidebar + topbar + mobile header	bardzo wysoka	**zaczerpnąć wzorzec**
Szybkie wyszukiwanie `Ctrl+K`	bardzo wysoka	**zaadaptować niemal wprost**
Rejestr modułowych sekcji projektu	bardzo wysoka	**silnie wykorzystać jako inspirację**
Status + postęp + dojrzałość	wysoka	**zaadaptować, ale rozdzielić od zdrowia projektu**
Timeline / aktualizacje projektu	wysoka	**zaadaptować do historii i checkpointów Centrum**
Zasoby / dokumentacja / repozytorium	wysoka	**zaadaptować do bogatszego modelu domenowego Centrum**

Element Po Kapiemu	Ocena dla Centrum	Decyzja
Port autoryzacji + role	wysoka później	**wzorzec do wykorzystania przy Hubie**
Express sesje + HttpOnly cookie + rate limit + Origin check	bardzo wysoka	**zaczerpnąć przy backendzie Hub**
`/api/health`	bardzo wysoka	**zaczerpnąć praktycznie wprost**
Docker Compose + healthchecki	bardzo wysoka	**zaczerpnąć przy Raspberry Pi Hub**
GitHub Actions quality gate	bardzo wysoka	**zaadaptować do Centrum**
Workflow publikacji	średnia	**użyć później dla kontrolowanego eksportu do Po Kapiemu**
Guest lock / supporter gating	niska	**odrzucić dla prywatnego Centrum**
Komentarze / głosowania / społeczność	bardzo niska	**odrzucić**
Recenzje / testy / porównania	brak związku	**odrzucić**
`siteData` / `projectDetails` jako statyczne źródło danych	negatywna	**nie kopiować**
localStorage jako biznesowe źródło prawdy	negatywna	**nie kopiować**

4.2. Wzorzec 1 — shell aplikacji i nawigacja

Po Kapiemu ma dojrzały układ:

```
Sidebar
  ├─ nawigacja główna
  ├─ obszar użytkownika
  ├─ akcja główna
  ├─ motyw
  └─ profil

Topbar
  ├─ przełącznik widoku
  ├─ szybkie wyszukiwanie
  └─ akcje kontekstowe
```

Dla Centrum nie należy kopiować treści menu, ale sam **model ergonomii** jest bardzo trafny.

Proponowany odpowiednik:

```
CENTRUM PROJEKTOWE
├─ Start
├─ Projekty
├─ Skrzynka
├─ Wymaga uwagi
├─ Decyzje
├─ Wyszukiwanie
├─ Zasoby
└─ Dane i kopie zapasowe
```

W topbarze:

- `Ctrl+K` — szybkie wyszukiwanie,
- `+ Szybki wpis` — główna akcja,
- status synchronizacji / huba — później,
- status kopii zapasowej — później.

4.3. Wzorzec 2 — szybkie wyszukiwanie

To jeden z najlepszych gotowych elementów Po Kapiemu.

W repozytorium istnieją:

- normalizacja tekstu,
- grupowanie wyników,
- obsługa klawiatury,
- `Ctrl+K`, `Esc`, strzałki i `Enter`,
- pełny ekran wyników,
- podział publiczny/redakcyjny.

Dla Centrum warto zachować mechanikę interakcji, ale źródła wyników muszą zostać zmienione na encje Centrum:

- Projekty,
- Wpisy,
- Decyzje,
- Elementy pracy,
- Pytania,
- Blokady,
- Dokumenty,
- Repozytoria,
- Zasoby,
- przyszłe pliki.

Nie należy kopiować indeksu opartego o statyczne `siteData`.

4.4. Wzorzec 3 — rejestr sekcji projektu

`rejestrSekcjiProjektu.tsx` w Po Kapiemu to bardzo wartościowy wzorzec: sekcja posiada identyfikator, etykietę, warunek widoczności i renderowaną zawartość. Dzięki temu karta projektu jest modułowa, a nie jednym monolitycznym komponentem.

Dla Centrum proponuje się analogiczny rejestr:

```
Podsumowanie
Gdzie skończyłem?
Stan wejściowy
Decyzje
Realizacja
Otwarte pytania
Blokady
Dokumentacja
Pliki
Repozytoria
Zasoby
Historia
Zdrowie projektu
Integracje
```

Nie należy jednak kopiować lokalnego drag-and-drop opartego o localStorage jako źródło biznesowego stanu. Preferencja układu może być lokalna, ale dane domenowe muszą pozostawać w repozytorium danych Centrum.

4.5. Wzorzec 4 — status, postęp i dojrzałość

Po Kapiemu rozdziela:

- status projektu,
- postęp procentowy,
- dojrzałość,
- kamienie milowe,
- przyszłe etapy,
- sugestię dojrzałości.

Dla Centrum warto przejąć tę ideę, ale rozszerzyć ją o czwarty wymiar:

Zdrowie projektu.

Docelowo:

Wymiar	Odpowiada na pytanie
Status	w jakim trybie znajduje się projekt?
Dojrzałość	jak daleko od pomysłu do stabilnego rozwiązania?

Wymiar	Odpowiada na pytanie
Postęp	ile zaplanowanej pracy wykonano?
Zdrowie	czy projekt jest spójny, aktualny i odblokowany?

Nie wolno tych wartości sklejać w jeden procent.

4.6. Wzorzec 5 — backend auth i bezpieczeństwo

Po Kapiemu posiada już sprawdzony wzorzec, który można później przenieść koncepcyjnie do Centrum Hub:

- Express,
- `helmet`,
- rate limit dla logowania,
- hasła hashowane bcrypt,
- losowy token sesji,
- w bazie jedynie SHA-256 tokenu,
- `HttpOnly`, `SameSite=Strict`,
- `Secure` w produkcji,
- czyszczenie wygasłych sesji,
- sprawdzanie `Origin` dla mutacji,
- idempotentne wylogowanie,
- generyczny komunikat błędnego logowania.

Warto zaczerpnąć wzorzec, nie kopiować bezrefleksyjnie kod 1:1. Centrum może mieć prostszy model użytkownika, ale zabezpieczenia sesji są dobre.

4.7. Wzorzec 6 — Docker, healthcheck i CI

Po Kapiemu pokazuje gotowy model:

```
PostgreSQL
   ↓ healthcheck
Aplikacja
   ↓ healthcheck /api/health
Caddy
```

oraz quality gate GitHub Actions:

- instalacja zależności,
- kontrola kodowania,
- lint,

- testy,
- build,
- sprawdzenie serwera.

Dla Raspberry Pi Hub warto zachować ten styl operacyjny:

- Docker Compose,
- restart `unless-stopped`,
- healthchecki,
- czytelne wolumeny,
- brak publicznego wystawienia bazy,
- automatyczny test build/test w CI.

Caddy jako publiczny reverse proxy **nie jest obowiązkowy**, jeżeli Centrum będzie dostępne wyłącznie przez Tailscale.

4.8. Wzorzec 7 — Ogarniacz: outbox, retry i idempotencja

Mapa Po Kapiemu → Ogarniacz jest bardzo wartościowa także dla Centrum. Wskazuje właściwą kolejność:

```
ręczne polecenie użytkownika
        ↓
wspólny kontrakt
        ↓
outbox
        ↓
retry + deduplikacja
        ↓
API / bridge
        ↓
Ogarniacz
```

Najważniejsze elementy do przejęcia:

- `sourceApp`,
- `sourceType`,
- `sourceId`,
- `sourceUrl`,
- `title`,
- `description`,
- `dueAt`,
- `remindAt`,
- `priority`,

- `target`,
- `idempotencyKey`,
- `metadata`,
- status integracji,
- brak automatycznego wysyłania wszystkiego,
- synchronizacja zwrotna dopiero później.

To jest jeden z najlepszych elementów całej architektury Po Kapiemu i warto go potraktować jako wspólny standard integracyjny całego ekosystemu.

---

5. Co należy odrzucić albo tylko potraktować jako inspirację

5.1. Funkcje społecznościowe

Nie przenosić do Centrum:

- komentarzy społeczności,
- głosowań,
- obserwujących,
- supporter gating,
- guest lock,
- społecznościowych projektów użytkowników.

Centrum ma być prywatnym narzędziem operacyjnym, nie portalem społecznościowym.

5.2. Recenzje, testy i porównania

Są specyficzne dla Po Kapiemu. W Centrum mogą pojawić się jedynie jako **typ projektu lub dokument**, a nie jako moduł systemowy.

5.3. Statyczne `siteData` i `projectDetails`

Po Kapiemu nadal korzysta z rozbudowanych danych statycznych w kodzie. To rozsądne dla prototypu publicznej witryny, ale w Centrum byłoby krokiem wstecz.

Centrum powinno rozwijać:

```
modele domenowe
→ repozytoria danych
→ IndexedDB
→ później Hub API / sync
```

Nie:

```
hardcoded TypeScript data
→ komponenty
```

5.4. localStorage jako źródło prawdy

W Po Kapiemu localStorage jest używany m.in. dla bookmarków, układu sekcji i dojrzałości. W Centrum może pozostać jedynie dla preferencji prezentacyjnych:

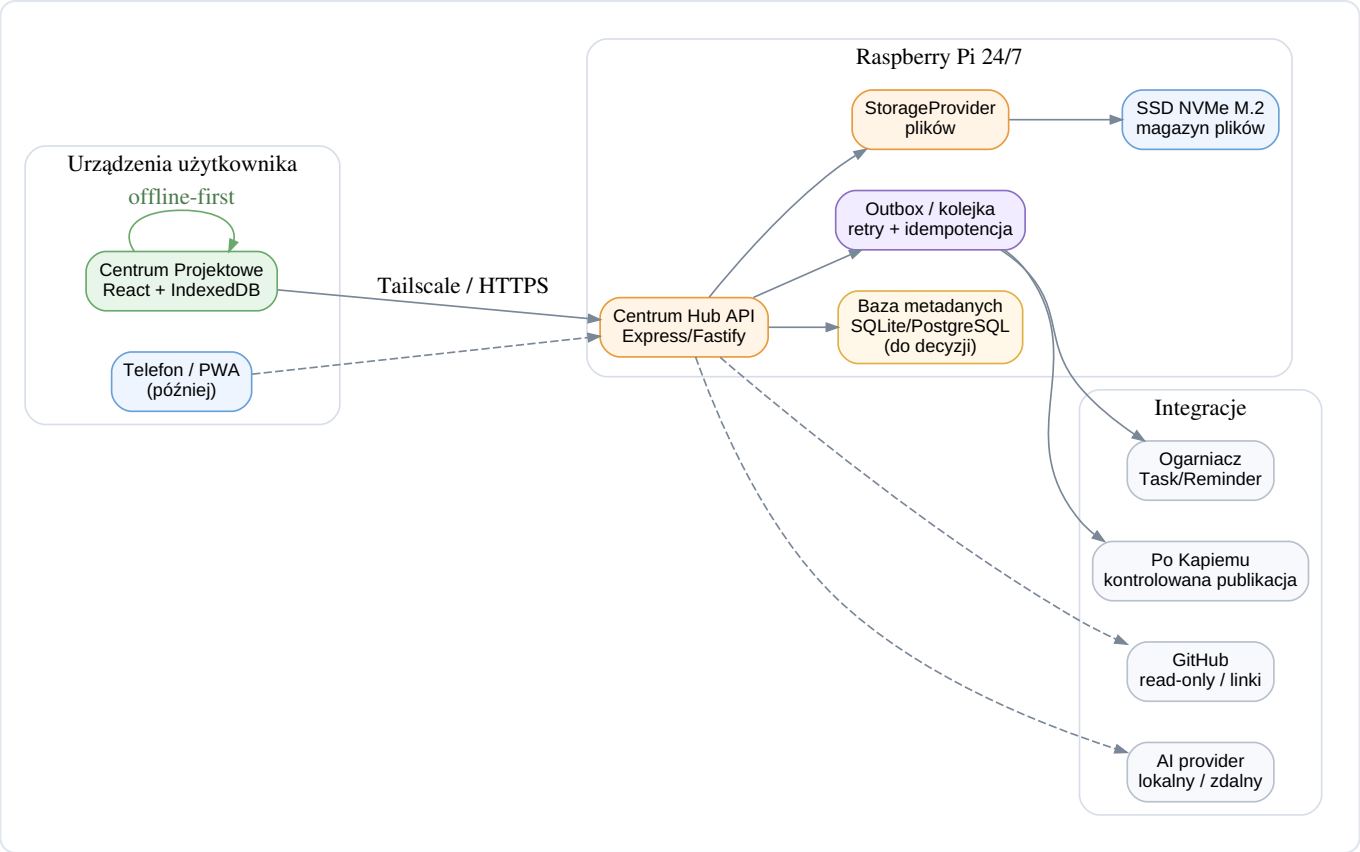
- motyw,
- gęstość interfejsu,
- układ paneli,
- ostatnio otwarta sekcja.

Nie dla:

- decyzji,
- dokumentów,
- statusu realizacji,
- relacji,
- historii,
- zdrowia projektu.

5.5. Publiczny Caddy + domena jako wymóg

Dla Centrum Hub domyślny model ma być prywatny. Jeżeli wybierzemy Tailscale-only, publiczna domena, porty 80/443 i publiczny certyfikat nie są konieczne w pierwszym wdrożeniu.

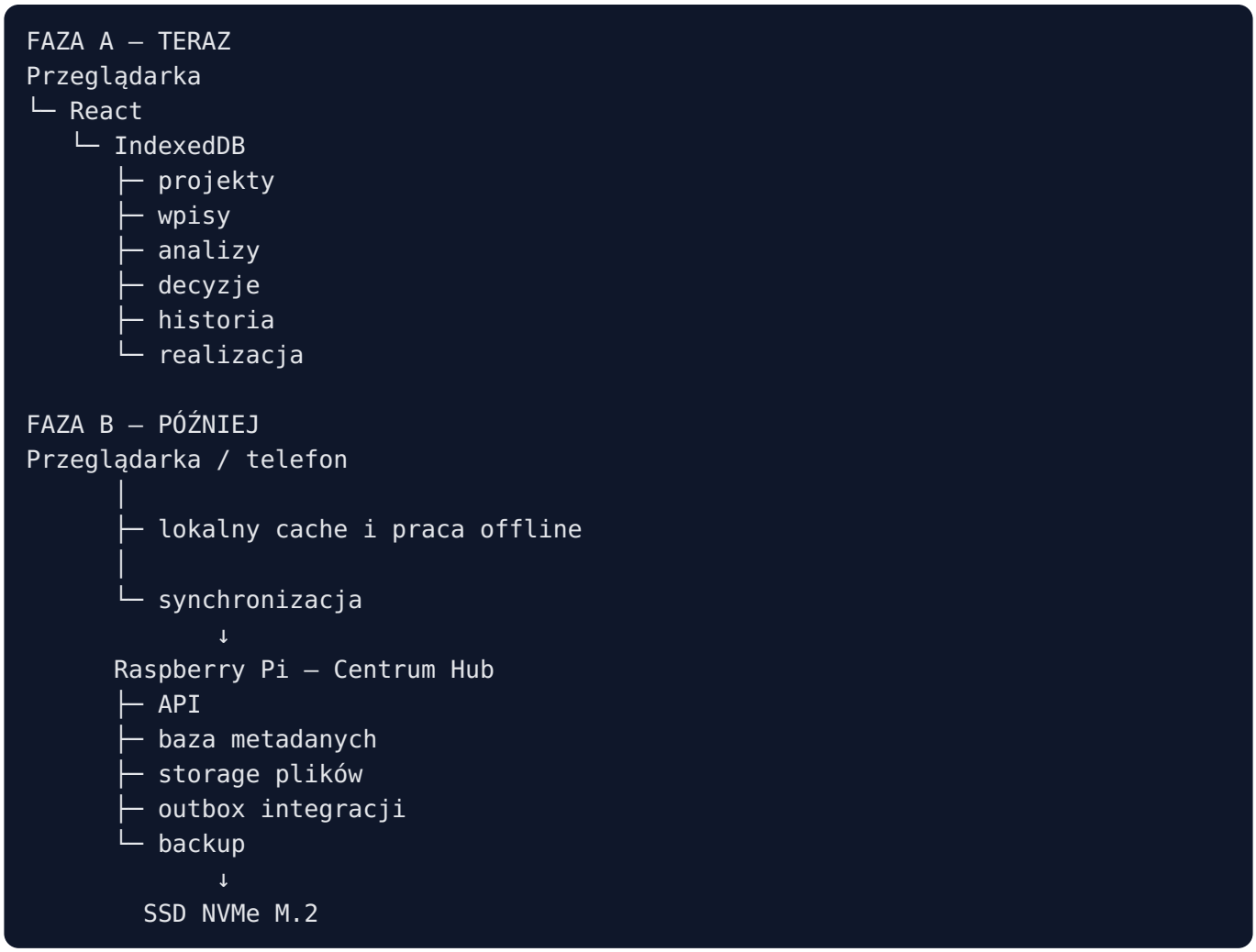


Docelowy kierunek: local-first klient + prywatny Centrum Hub na Raspberry Pi + SSD NVMe + kontrolowane integracje.

6. Nowa architektura docelowa Centrum Projektowego

6.1. Zasada nadrzędna

Architektura ma umożliwiać **działanie bez Raspberry Pi**, a następnie bezpieczne rozszerzenie o prywatny Hub.



6.2. Warstwy

A. Klient Centrum

- React + TypeScript,
- IndexedDB,
- UI i domena,
- możliwość pracy offline,
- szybki zapis bez wymagania sieci.

B. Centrum Hub

Planowana usługa na Raspberry Pi:

- API prywatne,
- synchronizacja danych,
- przechowywanie metadanych plików,
- upload/download,
- outbox integracji,

- endpoint zdrowia,
- mechanizm autoryzacji,
- backupy.

C. Magazyn plików

SSD NVMe M.2 podłączony do Raspberry Pi.

Nie przechowywać dużych plików jako blobów w IndexedDB.

D. Integracje

- GitHub,
- Ogarniacz,
- Po Kapiemu,
- AI providers,
- opcjonalni zewnętrzni providerzy storage.

---

7. Model plików i storage — odejście od Google Drive jako założenia

7.1. Google Drive nie może być typem domenowym pierwszej klasy

Wcześniejszy model `Document.type = GOOGLE_DRIVE` był zbyt konkretny.

Nowy model powinien rozdzielać:

```
Dokument / Plik domenowy
        ↓
StorageLocation
        ↓
StorageProvider
```

Przykład:

```
PLIK-0027
Nazwa: mechanizm-v4.step
Projekt: Stojak na słuchawki
Kategoria: CAD
Status: Aktualny
Wersja: 4
Provider: HUB_FILE_SYSTEM
Ścieżka logiczna: /projekty/.../
Rozmiar: ...
SHA-256: ...
```

7.2. Proponowany model `StorageProvider`

Pierwsze typy:

- `LOCAL_REFERENCE` — plik istnieje lokalnie, Centrum zna tylko metadane,
- `HUB_FILE_SYSTEM` — plik znajduje się na Raspberry Pi / SSD,
- `EXTERNAL_LINK` — zewnętrzne łącze,
- `GITHUB` — repozytorium lub plik GitHub,
- `OTHER` — rozszerzenie przyszłościowe.

Opcjonalnie później:

- Seafile,
- Nextcloud,
- S3-compatible storage,
- Google Drive,
- OneDrive.

Żaden z nich nie powinien być wymagany do działania aplikacji.

7.3. Metadane pliku

Minimum:

- `id`,
- `projectId`,
- `name`,
- `logicalPath`,
- `mimeType`,
- `size`,
- `checksumSha256`,
- `storageProviderId`,
- `storageObjectKey`,

- `version`,
- `status`,
- `privacy`,
- `createdAt`,
- `updatedAt`,
- `source`,
- opcjonalne relacje do Area / Stage / Decision / WorkItem.

7.4. Plik ≠ dokumentacja

Warto rozdzielić:

- **Plik** — fizyczny obiekt binarny lub referencja,
- **Dokument** — pozycja wiedzy projektowej mogąca wskazywać plik, URL, repozytorium albo treść wewnętrzną.

Dzięki temu jeden plik może być używany przez kilka dokumentów, a dokument może istnieć bez fizycznego pliku.

---

8. Raspberry Pi jako przyszły Centrum Hub

8.1. Rola

Najbardziej przyszłościowy wariant:

```
Raspberry Pi 24/7
├─ Centrum Hub API
├─ baza metadanych
├─ usługa storage
├─ SSD NVMe
├─ outbox integracji
├─ automatyczne backupy
└─ prywatny dostęp przez Tailscale
```

Nie oznacza to, że Raspberry Pi ma stać się obowiązkowe natychmiast.

8.2. Co warto zaczerpnąć z backendu Po Kapiemu

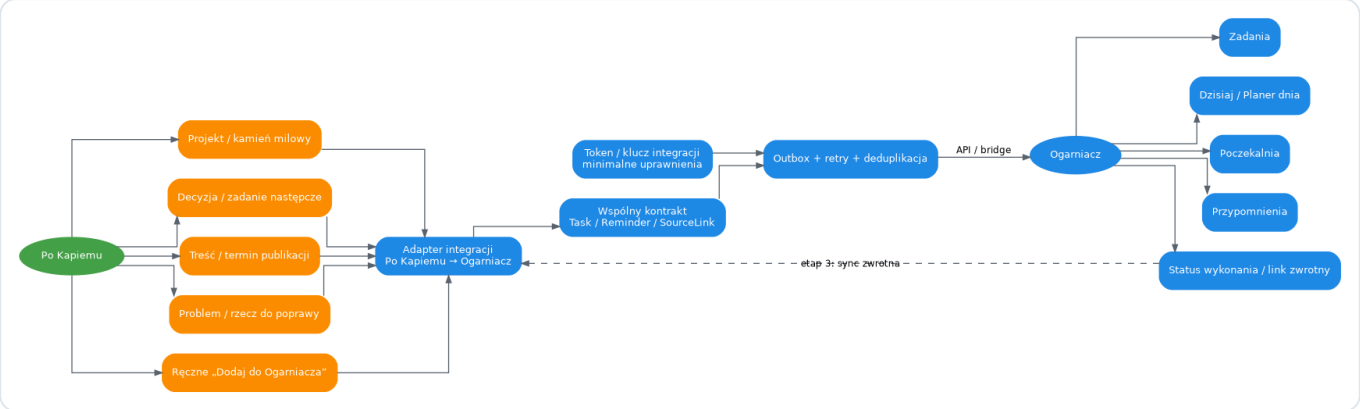
Przy budowie Hub warto wykorzystać jako punkt odniesienia:

- Express application factory,
- health endpoint,
- bezpieczne sesje,
- `helmet`,
- rate limiting,
- kontrolę `Origin`,
- Docker Compose,
- oddzielenie aplikacji od bazy,
- wolumeny,
- healthcheck zależności,
- 1.

8.3. Czego nie kopiować automatycznie

- pełnego systemu ról Po Kapiemu,
- publicznej domeny i Caddy jako obowiązkowego elementu,
- PostgreSQL tylko dlatego, że już występuje w Po Kapiemu,
- modelu backendu, który zna wyłącznie użytkowników i sesje.

Centrum Hub musi być projektowany **od domeny Centrum**, nie od infrastruktury Po Kapiemu.



Po Kapiemu → Ogarniacz — istniejący wzorzec outbox, retry, idempotencji i etapowej synchronizacji, który warto uogólnić dla Centrum.

9. Zasady integracji z Ogarniaczem i Po Kapiemu

9.1. Ogarniacz

Centrum powinno eksportować do Ogarniacza jedynie elementy wykonawcze:

- zadanie,
- przypomnienie,
- follow-up,
- termin,
- oczekiwanie wymagające przypomnienia.

Nie eksportować całej wiedzy projektowej.

Model:

```
ElementPracy / Decyzja / Blokada
         ↓ użytkownik wybiera
„Wyślij do Ogarniacza”
         ↓
kontrakt integracji
         ↓
outbox
         ↓
retry + idempotencyKey
         ↓
Ogarniacz
```

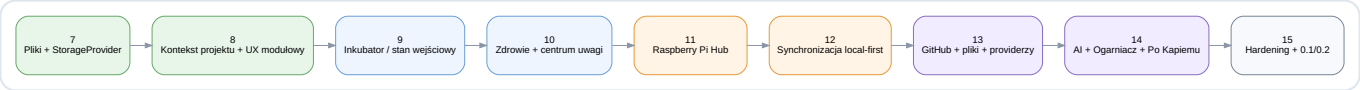
9.2. Po Kapiemu

Integracja odwrotna:

```
Centrum Projektowe
     ↓ jawna selekcja
PublicationSelection
     ↓ preview
„Te informacje opuszczą Centrum”
     ↓
Po Kapiemu
```

`PRIVATE` — nie może opuścić Centrum.
`INTERNAL` — wymaga jawnej zmiany klasyfikacji.
`SHAREABLE` — może zostać wybrane.

Nie tworzyć automatycznego „Opublikuj cały projekt”.



Zaktualizowana sekwencja etapów 7–15 po uwzględnieniu storage i Raspberry Pi Hub.

10. Zaktualizowany etapowy plan prac 7–15

Etap 7 — Pliki, dokumenty i abstrakcja StorageProvider

Cel: przygotować domenę plików pod Raspberry Pi bez budowania jeszcze serwera.

Zakres:

- `FileAsset` / `Plik`,
- `Document` / `Dokument`,
- `StorageProvider`,
- lokalne referencje,
- zewnętrzne linki,
- GitHub jako referencja,
- wersjonowanie pliku,
- checksum,
- klasyfikacja prywatności,
- relacje do projektu/obszaru/etapu/decyzji,
- backup nowych encji.

Inspiracja z Po Kapiemu: `PlikProjektu`, `DokumentacjaProjektu`, bezpieczne URL, załączniki aktualizacji.
Nie kopiować: pojedynczego `downloadUrl` jako całego modelu storage.

Etap 8 — Modułowa karta projektu, zasoby, kompetencje i globalne wyszukiwanie

Cel: wykorzystać najlepsze wzorce UX Po Kapiemu.

Zakres:

- rejestr sekcji projektu,
- stałe domenowe sekcje + widoczność zależna od danych,
- możliwość lokalnego ustawienia kolejności paneli,
- zasoby sprzętowe / programowe / usługi,
- kompetencje,
- relacje projektów,
- `Ctrl+K`,
- globalne wyszukiwanie po głównych encjach.

Inspiracja z Po Kapiemu: `rejestrSekcjiProjektu`, `SzybkieWyszukiwanie`, `Layout`.
Zasada: kopiować wzorzec interakcji, nie publiczne menu ani statyczne dane.

Etap 9 — Stan wejściowy projektu / Inkubator

Cel: zamieniać krótki pomysł w uporządkowany model bez wymagania natychmiastowej pracy użytkownika.

Zakres:

- typ projektu,
- dojrzałość,
- cel,
- rezultat docelowy,
- co wiemy,
- czego nie wiemy,
- założenia,
- decyzje do podjęcia,
- możliwe ścieżki,
- ryzyka,
- zależności,
- następny sensowny krok,
- review użytkownika.

Inspiracja z Po Kapiemu: dojrzałość, kamienie i sugestia dojrzałości.
Różnica: w Centrum to model analityczny, nie element publicznego statusu.

Etap 10 — Karta zdrowia + Centrum uwagi

Cel: pokazać nie tylko „ile zrobiono”, lecz „czy projekt jest zdrowy”.

Zakres:

- ustalenia,
- realizacja,
- blokady,
- dokumentacja,
- aktualność,
- organizacja,
- ręczna korekta z powodem,
- osobny postęp realizacji,
- ekran „Wymaga uwagi”,
- panel decyzji oczekujących.

Inspiracja z Po Kapiemu: dashboard, status ring, karty aktywności.
Nie kopiować: statusów społecznościowych i statycznych liczników.

Etap 11 — Raspberry Pi Hub — fundament backendu

Cel: uruchomić prywatny serwer Centrum, ale jeszcze bez pełnej synchronizacji.

Zakres:

- osobny katalog / usługa backend,
- `/api/health`,
- auth,
- storage API,
- metadane plików,
- zapis na SSD,
- Docker Compose,
- wolumeny,
- healthchecki,
- Tailscale-only jako domyślny tryb dostępu,
- 1.

Inspiracja z Po Kapiemu: Express auth, health, Docker Compose, quality workflow.
Decyzja otwarta: SQLite vs PostgreSQL; Express vs inny lekki framework.

Etap 12 — Synchronizacja local-first ↔ Hub

Cel: drugi komputer lub telefon nie może wymusić przejścia na „online-only”.

Zakres:

- dziennik zmian,
- stabilne ID,
- wersjonowanie rekordów,
- outbox synchronizacji,
- retry,
- idempotencja,
- konflikt jawny,
- brak last-write-wins dla decyzji i krytycznych encji,
- status synchronizacji per rekord / projekt,
- tryb offline.

Inspiracja z Po Kapiemu: outbox + retry + idempotencyKey z planu integracji Ogarniacza.
Nie kopiować: ukrytej dwukierunkowej synchronizacji.

Etap 13 — GitHub i providerzy zewnętrzni

Cel: źródła zewnętrzne dostarczają kontekst, ale nie przejmują domeny.

Zakres:

- read-only GitHub provider,
- branche i commity,
- ręczne powiązanie commit ↔ decyzja / element pracy,
- StorageProvider adapters,
- opcjonalny dostęp do plików poza Centrum,
- możliwość dodania Seafile/Nextcloud/S3 później.

Zasada: `commit != decyzja wdrożona`.

Etap 14 — AI + Ogarniacz + Po Kapiemu

Cel: wykorzystać gotowy rdzeń do bezpiecznych integracji.

Zakres:

AI

- wersjonowane analizy,
- RuleBased / Local LLM / Remote LLM,
- walidowany structured output,
- review przed zastosowaniem.

Ogarniacz

- ręczny eksport,
- outbox,
- retry,
- idempotencyKey,
- status integracji,
- dopiero później sync zwrotna.

Po Kapiemu

- PublicationSelection,
- privacy gate,
- preview,
- kontrakt danych / JSON,
- brak automatycznej publikacji.

Etap 15 — Hardening i wydanie

Cel: codzienna niezawodność zamiast dokładania kolejnych modułów.

Zakres:

- pełna regresja,
- real browser smoke test,
- backup/restore,
- upload/download plików,
- awaria Hub,
- praca offline,
- ponowne połączenie,
- konflikty,
- dostępność,
- polonizacja UI,
- security review,
- dokumentacja operacyjna Raspberry Pi,

- procedura backupu SSD,
- oznaczenie wydania.

---

11. Zasady techniczne i antycele

11.1. Zasady

1. **Local-first do czasu, aż synchronizacja będzie naprawdę gotowa.**
2. **Oryginał wpisu nigdy nie jest nadpisywany przez AI.**
3. **Analiza ≠ fakt ≠ decyzja.**
4. **Decyzja ≠ zadanie.**
5. **Plik ≠ dokument.**
6. **Zdrowie ≠ postęp.**
7. **Commit ≠ dowód wdrożenia.**
8. **Integracja zewnętrzna wymaga jawnego kontraktu.**
9. **Outbox i idempotencja przed automatyzacją.**
10. **Duże pliki nie trafiają do IndexedDB jako stałe źródło prawdy.**
11. **Google Drive, Nextcloud, Seafile itd. są providerami, nie fundamentem domeny.**
12. **Raspberry Pi może zniknąć z sieci bez utraty możliwości zapisania nowej myśli.**

11.2. Antycele

Nie budować teraz:

- publicznej społeczności,
- komentarzy,
- głosowań,
- CMS-a,
- rozbudowanej wielоużytkownikowości,
- płatności,
- publicznego API,
- pełnej synchronizacji przed zdefiniowaniem konfliktów,
- magicznego AI podejmującego decyzje bez review.

---

12. Kryteria gotowości

Po Etapie 8

Centrum powinno być bardzo wygodnym lokalnym narzędziem z:

- bezpiecznym backupem,
- dokumentami i plikami jako encjami,
- zasobami,
- modularną kartą projektu,
- szybkim wyszukiwaniem.

Po Etapie 10

Powinno działać jako prawdziwe centrum zarządzania:

- wie, czym jest projekt,
- wie, gdzie skończono,
- pokazuje zdrowie,
- pokazuje problemy przekrojowo,
- pomaga zdecydować, na czym skupić uwagę.

Po Etapie 12

Powinno działać wielourządzeniowo:

- offline-first,
- z prywatnym Hubem,
- z plikami na SSD,
- z jawnymi konfliktami,
- bez utraty danych podczas awarii sieci.

Po Etapie 15

Powinno nadawać się do stałego codziennego używania bez traktowania go jako eksperymentalnego prototypu.

13. Pytania decyzyjne — architektura danych i storage

P1. Jaką docelową rolę ma pełnić Raspberry Pi?

A. Tylko magazyn plików dla Centrum.
B. Magazyn plików + automatyczne kopie zapasowe.
C. Centralny prywatny **Centrum Hub**: pliki + API + backup + późniejsza synchronizacja urządzeń.

> **Sugerowane: C** — najlepiej wykorzystuje sprzęt 24/7 i daje bazę pod cały przyszły ekosystem, bez wymuszania rezygnacji z local-first.

Alternatywa	+ Zalety	- Wady
B	**prostsze uruchomienie; od razu poprawia bezpieczeństwo danych; mniejszy zakres backendu**	**dane Centrum i pliki nadal pozostają dwoma osobnymi światami**
C	**jeden prywatny hub; baza pod telefon, laptop, Ogarniacz i przyszłe integracje**	**większa odpowiedzialność za serwer, aktualizacje, backup i synchronizację**

P2. Co ma być źródłem prawdy dla danych domenowych podczas najbliższych etapów?

A. IndexedDB pozostaje źródłem prawdy, Hub dołącza później.
B. Raspberry Pi od razu staje się jedynym źródłem prawdy.
C. Od razu równorzędny model hybrydowy IndexedDB ↔ Hub.

> **Sugerowane: A** — nie zatrzymuje już działającego MVP i odkłada najtrudniejszy problem synchronizacji do osobnego etapu.

Alternatywa	+ Zalety	- Wady
B	prosty mentalnie centralny stan; łatwiejsze współdzielenie między urządzeniami	duży refaktor teraz; zależność działania od dostępności serwera
C	najlepsze doświadczenie offline w stanie docelowym	konflikty i synchronizacja stają się krytycznym problemem zanim aplikacja zdąży dojrzeć

P3. Jak fizycznie przechowywać pliki na Raspberry Pi?

A. Zwykły filesystem na SSD + lekkie własne API Centrum.
B. Seafile jako gotowa warstwa plikowa.
C. Nextcloud jako pełna prywatna chmura.

> **Sugerowane: A** — najmniej narzutu, pełna kontrola nad formatem i najlepsze dopasowanie do domeny Centrum; adaptery B/C można dodać później.

Alternatywa	+ Zalety	- Wady
B	gotowa synchronizacja plików, wersjonowanie, aplikacje klienckie	dodatkowa usługa i baza; Centrum musi integrować cudzy model plików
C	najbogatszy ekosystem, WebDAV, aplikacje mobilne i desktopowe	największy narzut operacyjny; wiele funkcji, których Centrum nie potrzebuje

P4. Jaką bazę metadanych docelowo uruchomić na Raspberry Pi Hub?

A. SQLite z WAL — jedna baza dla Centrum Hub.
B. PostgreSQL — wzorzec podobny do Po Kapiemu.
C. Bez osobnej bazy — tylko filesystem + pliki JSON.

> **Sugerowane: A na start** — dla prywatnej aplikacji jednej osoby upraszcza backup i administrację; Postgres pozostaje łatwym kierunkiem migracji, jeśli Hub rozrośnie się w usługę dla wielu aplikacji.

Alternatywa	+ Zalety	- Wady
B	**sprawdzony wzorzec z Po Kapiemu; świetne transakcje, indeksy i przyszła wielożytkownikowość**	**więcej usług, konfiguracji i backupu niż potrzeba na początku**
C	**minimalna liczba komponentów**	**słabe relacje, migracje, indeksowanie i spójność danych; szybki dług techniczny**

P5. Jak wdrażać usługi na Raspberry Pi?

A. Docker Compose + healthchecki + wolumeny.
B. Natywne usługi systemd na Raspberry Pi OS.
C. Kubernetes / k3s.

> **Sugerowane: A** — można bezpośrednio wykorzystać doświadczenia z Po Kapiemu, a przenoszenie i odtwarzanie środowiska jest prostsze.

Alternatywa	+ Zalety	- Wady
B	**mniejszy narzut runtime; bardzo dobre dla kilku prostych usług**	**więcej ręcznej konfiguracji hosta i trudniejsze odtworzenie środowiska 1:1**
C	**świetna orkiestracja i skalowanie**	**zdecydowany overkill dla prywatnego Raspberry Pi i niepotrzebne koszty poznawcze**

P6. Jak uzyskiwać dostęp do Hubu spoza domu?

A. Tailscale-only.
B. Publiczna domena + HTTPS + reverse proxy.
C. Tylko LAN, bez dostępu zdalnego.

> **Sugerowane: A** — najlepiej pasuje do prywatnego charakteru systemu i ogranicza powierzchnię ataku.

Alternatywa	+ Zalety	- Wady
B	**dostęp z dowolnej przeglądarki bez klienta VPN**	**większa odpowiedzialność za publiczne auth, TLS, monitoring i aktualizacje**
C	**najprostsze bezpieczeństwo sieciowe**	**brak wygodnego dostępu z telefonu/laptopa poza domem**

P7. Czy pliki z SSD mają być dostępne również poza interfejsem Centrum?

A. Nie — wyłącznie przez Centrum.
B. Tak — przez zwykły udział plikowy / SFTP / podobny dostęp.
C. Tak — pełna warstwa w stylu prywatnego Google Drive.

> **Sugerowane: B** — CAD, PDF, grafiki i inne materiały powinny dać się otwierać normalnymi programami bez uzależnienia od UI Centrum.

Alternatywa	+ Zalety	- Wady
B	**brak vendor lock-in; wygodna edycja plików; naturalny workflow CAD/grafika**	**Centrum musi wykrywać zmiany wykonane poza aplikacją**
C	**najbardziej kompletne doświadczenie użytkownika i gotowe klienty**	**praktycznie dokładamy osobny duży produkt do utrzymania**

---

14. Pytania decyzyjne — UX, bezpieczeństwo i integracje

P8. Jak daleko kopiować interfejsowe wzorce z Po Kapiemu?

A. Tylko inspiracja wizualna.
B. Przenieść wzorce shellu, nawigacji, Ctrl+K i modularnych sekcji, ale nie styl marki i funkcje publiczne.
C. Utrzymać możliwie wspólny Design System obu aplikacji.

> **Sugerowane: B** — daje największą oszczędność pracy bez zacierania różnicy między publicznym Po Kapiemu i prywatnym Centrum.

Alternatywa	+ Zalety	- Wady
B	**sprawdzone UX, szybsza implementacja, mniejsza liczba nowych decyzji projektowych**	**trzeba uważać, aby nie przenieść założeń społecznościowych i publicznych**
C	**spójność całego ekosystemu i łatwiejsze współdzielenie komponentów**	**silne sprzężenie repozytoriów i ryzyko, że potrzeby jednego produktu zaczną ograniczać drugi**

P9. Jaki model dostępu zastosować, gdy powstanie Hub?

A. Sam Tailscale, bez logowania do Centrum.
B. Tailscale + własna sesja użytkownika inspirowana Po Kapiemu.
C. Rozbudowany system kont i ról od początku.

> **Sugerowane: B** — dwie niezależne bariery ochrony prywatnych projektów bez budowania pełnej wielożytkownikowości.

Alternatywa	+ Zalety	- Wady
B	**lepsza ochrona zgubionego urządzenia; można zaczerpnąć gotowe wzorce sesji z Po Kapiemu**	**dochodzi zarządzanie sesjami i hasłem**
C	**gotowość na współpracowników i różne poziomy dostępu**	**duży zakres bez realnej potrzeby MVP; znacznie więcej ścieżek bezpieczeństwa**

P10. Jak traktować układ sekcji projektu?

A. Stały układ dla wszystkich projektów.
B. Rejestr modułów + ukrywanie pustych sekcji + lokalna możliwość zmiany kolejności.
C. Pełny edytor dashboardu drag-and-drop zapisujący układ w domenie.

> **Sugerowane: B** — wykorzystuje najlepszy wzorzec Po Kapiemu bez budowania osobnego kreatora dashboardów.

Alternatywa	+ Zalety	- Wady
B	**elastyczne karty projektów; mniej pustych paneli; preferencje mogą pozostać lokalne**	**trzeba pilnować stabilnych identyfikatorów sekcji**
C	**maksymalna personalizacja**	**dużo UI i edge-case'ów o małej wartości dla pamięci projektowej**

P11. Jak prowadzić integrację z Ogarniaczem?

A. Najpierw ręczne „Wyślij do Ogarniacza" + outbox + idempotencja.
B. Od razu automatyczne reguły dla terminów i blockerów.
C. Od razu pełna dwukierunkowa synchronizacja statusów.

> **Sugerowane: A** — dokładnie ten sam ostrożny kierunek wynika z architektury integracji Po Kapiemu.

Alternatywa	+ Zalety	- Wady
B	**mniej ręcznej pracy i szybsze przypomnienia**	**łatwo zaspamować Ogarniacz i tworzyć zadania, których użytkownik nie chciał**
C	**najbardziej zautomatyzowany ekosystem**	**najtrudniejsze konflikty i ryzyko ukrytego nadpisywania stanów**

P12. Jak realizować publikację do Po Kapiemu?

A. Jawny wybór pojedynczych pól i sekcji + podgląd przed eksportem.
B. Gotowe profile publikacji, np. „publiczny opis projektu”.
C. Automatyczna synchronizacja wybranych projektów.

> **Sugerowane: A na start, B jako późniejsze udogodnienie** — prywatność ma pierwszeństwo przed wygodą publikacji.

Alternatywa	+ Zalety	- Wady
B	**szybsze publikowanie często powtarzanego zestawu danych**	**profil może po czasie obejmować pola, których użytkownik nie pamięta i nie chce już ujawniać**
C	**najmniej pracy ręcznej**	**nieakceptowalne ryzyko przypadkowego ujawnienia prywatnych informacji projektowych**

P13. Jak wykonywać kopię SSD z Raspberry Pi?

A. Brak dodatkowej kopii — SSD jest wystarczający.
B. Automatyczna kopia na drugi lokalny nośnik / komputer.
C. Drugi lokalny backup + zaszyfrowana kopia off-site później.

> **Sugerowane: C docelowo, B jako pierwszy krok** — pojedynczy SSD nie jest kopią zapasową, a prywatne dane projektowe mogą mieć wysoką wartość.

Alternatywa	+ Zalety	- Wady
B	**proste, darmowe po zakupie nośnika, szybkie odtwarzanie**	**oba nośniki mogą zostać utracone przy jednym zdarzeniu fizycznym**
C	**ochrona także przed kradzieżą, pożarem i awarią całej lokalizacji**	**trzeba zarządzać szyfrowaniem, kluczami i limitem zewnętrznej przestrzeni**

P14. Jak podchodzić do AI na Raspberry Pi?

A. Raspberry Pi nie uruchamia AI; jest tylko Hubem danych.
B. Architektura dopuszcza lekkiego lokalnego providera, ale nie jest on wymagany.
C. Raspberry Pi ma być głównym serwerem AI Centrum.

> **Sugerowane: B** — nie zamyka drogi do lokalnej analizy, ale nie pozwala, aby wymagania modelu AI dyktowały architekturę storage i synchronizacji.

Alternatywa	+ Zalety	- Wady
B	**przyszłościowość bez zobowiązania; możliwość użycia lekkich modeli lub usług lokalnych**	**część abstrakcji może długo pozostać niewykorzystana**
C	**maksymalna prywatność i brak opłat za API**	**wydajność Pi ogranicza wybór modeli i może wymusić niepotrzebnie droższy sprzęt**

Źródła i materiały porównawcze

- Repozytorium Centrum Projektowego: `https://github.com/kapi1944/Centrum_Projektowe`
- Repozytorium Po Kapiemu: `https://github.com/kapi1944/Po_Kapiemu`

- „Po Kapiemu — mapa architektury i przepływów”, stan 2026-10-08
- Diagramy przekazane razem z dokumentem: przepływ widoków, struktura techniczna, drzewo logiczne, integracja z Ogarniaczem

---

Status dokumentu: obowiązujący kierunek roboczy do czasu udzielenia odpowiedzi na pytania P1–P14.
Po zatwierdzeniu odpowiedzi należy przygotować osobną, wykonawczą serię promptów Etap 7–15 dostosowaną do wybranej architektury.